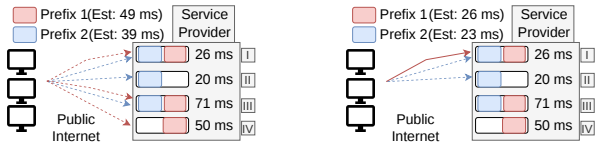


(a) (b)

Figure 4: SCULPTOR initially estimates latency from this UG to both the red and blue prefixes with the average over possible ingress latencies (4a). A priori, no prefix is advertised. SCULPTOR then advertises the red prefix and learns that ingress I is preferred over III and IV (4b). SCULPTOR uses this information to refine its latency estimate towards the blue prefix (ingress III is not a preferred option) without advertising the blue prefix, saving time.

3.4 Modeling & Measuring Routes

In Equation (2) we compute expectations over a random process that represents the routing, R, since we cannot exhaustively measure R. Specifically, SCULPTOR models UG paths probabilistically and improves its model over time through relatively few measurements.

3.4.1 Routing Model

A priori, for a given UG towards a given prefix, SCULPTOR treats all reachable ingress link options to which that prefix is advertised as equally likely. One could incorporate prior information about ingress preferences (from economic/routing models, for example), but such complexity is unnecessary for good performance.

We instead learn preferences over time via BGP measurements and observe which ingress the UG takes (§3.4.2). Upon learning that one ingress is preferred over the others receiving that prefix advertisement, we exclude that less-preferred ingress as an option for that UG in all future calculations for all prefixes for which both ingresses are an option. Prior work used a similar routing model [16, 66, 1] but did not extend it to deal with general objectives nor did it treat routing probabilistically.

As we learn which paths UGs prefer the most, our estimate of G on unmeasured configurations concentrates on the true value, allowing us to more efficiently carry out Equation (2). An example of this learning process is shown in Figure 4—SCULPTOR refines a latency estimate towards an unmeasured prefix (blue) using measurements towards other prefixes (red). Even though SCULPTOR has 50% confidence in which path the user takes (*i.e.,* in $R(A)$), SCULPTOR knows that the user's latency towards the blue prefix will be about 23 ms (*i.e.,* either 26 ms or 20 ms) and so SCULPTOR obtains a better estimate of the objective.

3.4.2 Refining the Model

Probabilistic estimates of G could be inaccurate until we learn UG preferences. To refine its model, SCULPTOR periodically measures $R(A)$ on its current configuration (§3.5) by advertising prefixes and measuring UG preferences by observing which routes users take. To use its BGP measurement budget wisely, SCULPTOR makes two design decisions:

What to measure: SCULPTOR measures the *current* configuration (as opposed to random or slightly different ones) since it uses a descent-based optimization (§3.5). This type of optimization compares the current advertisement to many adjacent/different ones, so knowing the current value is much more important than knowing adjacent ones.

When to measure: SCULPTOR measures either when (a) its uncertainty (according to Monte Carlo simulations, §3.6) exceeds a certain threshold or (b) SCULPTOR has not executed a measurement in many learning iterations. There are many possible choices of "uncertainty", we choose entropy [76]. Intelligently choosing when to measure offers a slight performance improvement over a fixed measurement schedule (§5.4.1).

3.5 Searching Over Configurations

We now describe the outer optimization loop in Equation (2) over advertisement configurations, A.

3.5.1 Initializing A

We initialize our configuration to be anycast on one prefix and unicast on remaining prefixes (*i.e.,* one prefix per site), since both are well-understood configurations with complementary strengths (§2.2). Remaining prefixes in the budget (if they exist) are advertised to random subsets of ingresses.

3.5.2 Gradient Descent

For the reasons discussed in Section 3.3 we cannot find A simply using an off-the-shelf solver. A reasonable choice to solve such problems are descent-based algorithms—ones that compare adjacent options to the current best guess and pick an adjacent option as the next configuration. However, an *additional* challenge in our setting is that we only have noisy estimates of the objective, so placing too much trust in objective estimates of adjacent configurations can lead to poor convergence.

SCULPTOR instead uses *gradient* descent since gradient descent slowly incorporates and remembers information about how useful each advertisement is rather than overcommitting, which is what greedy and random searches used by prior systems do [16, 1].

To apply gradient descent to our problem, we create a real-valued auxiliary variable to A, $\tilde{A}$, with entries between 0 and 1 representing "confidence" values that an advertisement to the

corresponding ingress will be beneficial. We threshold $\tilde{A}$ at 0.5 to translate real-valued confidences to whether or not we should advertise a prefix at an ingress (*i.e.,* thresholded $\tilde{A} = A$). For coordinate (p, l), SCULPTOR evaluates the inner objective (§3.6) with $A(p,l)$ set to 1 and then to 0, holding the rest of the (thresholded) configuration fixed. The difference between these two values, scaled by the derivative of a sigmoid at $\tilde{A}$, is the gradient entry (similar to prior work [77]). Since there are too many gradients to compute, we subsample gradient entries and track the largest ones (as in prior work [78]).

We implement SCULPTOR with Nesterov's Accelerated Gradient Descent (§3.5.2) in Python [79]. We use RMSProp with an initial learning rate of 1.0.

Scaling: Gradient descent scales with the product of the number of ingresses and number of prefixes but individual entry computation is independent and therefore parallelizable.

Convergence: Gradient descent converges to a local minimum for bounded objectives [80]. Our evaluations show that SCULPTOR finds good solutions over a wide range of emulated topologies (**??**) and converges quickly with thousands of UGs and ⟨peering, prefix⟩ pairs (§5). Allocating higher prefix budgets and adding richer advertisement capabilities (*e.g.,* BGP community tagging) can lead to convergence to a better minimum, which is an area for future work.

3.5.3 Termination

SCULPTOR stops gradient descent either when the BGP measurement budget (*i.e.,* time budget) has been exhausted or the average change in objective over iterations falls below a small threshold. We stop the optimization when measurement budget has been exhausted since SCULPTOR may make poor decisions if it has a bad estimate of the routing function.

3.6 Allocating Traffic

This section specifies the inner minimization over traffic allocations in Equation (2).

Given our routing model (§3.4.1) and current advertisement (§3.5), we Monte Carlo sample routing functions R. Each sample independently draws, for every UG and prefix, one ingress uniformly from the ingresses the model still considers possible for that pair (§3.4.1). For each sample we then solve the traffic allocation exactly with a general-purpose solver, and the expectation in Equation (2) is the average of those solutions. In practice we find that a few Monte Carlo samples per gradient entry is sufficient.

Given a sampled routing realization, solving for the optimal traffic allocation is then a problem in classical (*i.e.,* intradomain) traffic engineering. We solve traffic allocations (§3.6) with SciPy HIGHS [81].

Scaling: This component scales with the number of Monte Carlo samples and the scaling behavior of each traffic allocation, which depends on the objective, G.

Convergence: depends on the objective, G. SCULPTOR accepts any G for which, given fixed routes, the allocation problem is efficiently solvable. All objectives that we evaluate in Section 5.1.2 are linear programs.

4 Implementation

We prototype SCULPTOR on the PEERING testbed [21], which is available at 32 Vultr cloud sites. We describe how we built SCULPTOR on the real Internet and how we *emulate* a Service Provider for more thorough evaluation.

In practice, Service Providers can use SCULPTOR *safely* by only shifting production traffic onto prefixes after SCULPTOR's announcement exploration has converged and by integrating SCULPTOR into automated route configuration frameworks. More details about deployment details are in Appendix B.

4.1 Evaluation Limitations

We are not a Service Provider and so could not obtain actual volumes/capacities—latency and routes are the real Internet phenomenon that we, as researchers, are able to measure. We *do* evaluate SCULPTOR on hundreds of routing and capacity provisioning scenarios to demonstrate that its benefits are not particular to any deployment. Prior interdomain routing work that actually measures congestion *was conducted by Service Providers* [82, 83, 5, 59, 84, 85, 35, 6], whereas prior work from academics used similar methodology to our own [86, 53, 87].

4.2 On-Internet Deployments

We assess how SCULPTOR performs on the Internet using RIPE Atlas probes [88], which represent a subset of all clients. RIPE Atlas allows us to measure paths (and thus ingress links) to prefixes we announce from PEERING, which SCULPTOR needs to refine its model (§3). We limit the scale of our deployment to 10 sites to avoid reaching RIPE Atlas daily probing limits (15k traceroutes/day). These 10 sites include major hubs across North America, Europe, and South America. Choosing RIPE Atlas probes to maximize network coverage and geographic diversity, we select probes from 972 networks in 38 countries which have paths to 484 unique ingresses. Each probe has paths via approximately 60 ingresses. We use 12 prefixes and change advertisements at most once per hour to avoid route-flap-dampening.

We measure which ingress link that a probe took to a prefix using traceroutes. Specifically, we perform IP to AS mappings on traceroutes and identify the previous AS in the path to Vultr. In cases where we cannot infer the ingress link from a traceroute, we use the closest latency from the traceroute to the clients' (known) possible ingresses. For example, if an uninformative traceroute's latency was 40 ms to Vultr's Atlanta site and a client was known to have a 40 ms path through

AS1299 at that site, we say the ingress link was AS1299 at Atlanta.

4.3 Emulated Deployments

To more thoroughly evaluate SCULPTOR on more sites, we emulate hundreds of deployments using measurements from more networks and using thousands of random routing scenarios and volume distributions. These emulations are seeded with real latency measurements from targets to Vultr.

Selecting Targets and Measuring Latencies. We measure actual latency from IP addresses to our PEERING prototype as in prior work [1], selecting targets according to assumptions about Vultr cloud's client base.

We first tabulate a list of 5M IPv4 targets that respond to ping via probing each /24. Vultr informs cloud customers of which prefixes are reachable via which peers, and we use this information to tabulate a list of peers and clients reachable through those peers. We then measure latency from all clients to each peer individually by advertising a prefix solely to that peer using Vultr's BGP action communities and pinging clients from Vultr. We also measure performance from all clients to all providers individually, as providers provide global reachability.

In our evaluations, we limit our focus to clients who had a route through at least one of Vultr's direct peers (we exclude route server peers [89]). Vultr likely peers with networks with which it exchanges a significant amount of traffic [35], so clients with routes through those peers are more likely to be "important" to Vultr. We found 700k /24s with routes through 1086 of Vultr's direct ingresses. We removed clients whose lowest latency to Vultr was 1 ms or less, as these were assumed to be addresses related to infrastructure, leaving us with measurements from 666k /24s to 825 Vultr ingresses.

Emulating Traffic Distributions. We emulate traffic distributions in two ways: First, we randomly choose the total traffic volume of a site as a number between 1 and 10 and then divide volume up randomly among clients that anycast routes to that site. Client volumes in a site are set to be within one order of magnitude of each other. We avoid Zipf-like distributions as they lead to trivial solutions where a small fraction of sites dominate importance.

Second, we derive realistic volumes using APNIC AS populations [90], shown to be representative of Service Provider traffic volumes [91]. We assign each AS its aggregate population and divide it evenly among its clients. Our measurements represent 91% of APNIC volume.

Random deployment subsampling. We randomly subsample sites and targets to create diversity across emulations. The distribution of the number of /24s per peer is Pareto-like, so we consider random subsets of /24s through each ingress in a way that balances the number of unique /24s per ingress. Over *all* scenarios, we consider paths from clients in 52k prefixes to 873 ingresses. We use one tenth of the number of ingresses as the number of prefixes in our budget (10 prefixes for 3 site deployments, 60 prefixes for 32 site deployments), but SCULPTOR's benefits are not specific to a prefix budget (§5.4.2). Prior work found that using tens of prefixes to improve performance was a reasonable cost [16, 1].

4.4 Setting Resource Capacities

Emulations. The second method we use to infer ingress links is emulating user paths by assuming we know all user routing preferences (§3.2). We use a preference model where clients prefer peers over providers, and clients have a preferred provider. When choosing among multiple ingresses for the same peer/provider, clients prefer the lowest-latency option. We also add in random violations of the model. This approach allows us to evaluate our model on *all* client networks but may not represent actual routing conditions, though prior work found it held in 90% of the cases they studied. We conducted a sensitivity analysis on this model/assumption and found that SCULPTOR's performance gains are robust to variations (*e.g.,* random preferences).

We then assign resource capacities as some overprovisioned percentage of anycast catchment. (Discussions with operators from Service Providers suggested that they overprovision using this principle.) We report results for an overprovisioning rate of 10%, but find similar takeaways for 10% through 30%.

5 Evaluation

5.1 Evaluation Setting

5.1.1 Baseline Solutions

anycast.: A single prefix announcement to all peers/providers at all sites, which is a common configuration used by Service Providers today [92, 93, 46, 94, 95, 96, 49].

unicast.: A single prefix announcement to all peers/providers at each site (one per site). Another common configuration used in today's deployments [50, 51, 52].

AnyOpt & PAINTER.: Two proposed strategies for reducing steady-state latency compared to anycast [16, 1]. AnyOpt was one of the first works to systematically optimize ingress traffic, while PAINTER built on that effort. (We do not compute the solution for AnyOpt for our evaluations on the Internet since AnyOpt performs poorly in emulations, and since AnyOpt takes a long time to compute.)

One-per-Peering.: A unique prefix advertisement to each peer/provider, so many possible paths are always available from users. This solution serves as our performance upper-bound, even though it is prohibitively expensive. (We do not know an optimal solution with fewer prefixes.)

We compute both average overall latency and the fraction of traffic within 10 ms (very little routing inefficiency), 50

ms (some routing inefficiency), and 100 ms (lots of routing inefficiency) of the One-per-Peering solution for each advertisement configuration, as these statistics provide a more informative measure of latency improvement than averages.

5.1.2 Objectives Evaluated

While our approach likely accommodates other objectives [59, 35, 97, 1, 16, 52, 20, 17, 18, 98], it provides practical utility at the expense of a formal characterization of feasible functions, similar to prior gradient descent work. Objectives are described here on a high-level and formally written down in the Section A.2. Some objectives have multiple terms that trade-off, for example, lower steady-state latency for less congestion during site failures via constant multipliers. In evaluations, what makes one method solution method (§5.1.1) better than another is that the weighted numerical combination is lower and it is up to Service Providers to set these constants according to deployment goals. (We find that SCULPTOR individually beats other solutions on all metrics regardless of weighting.)

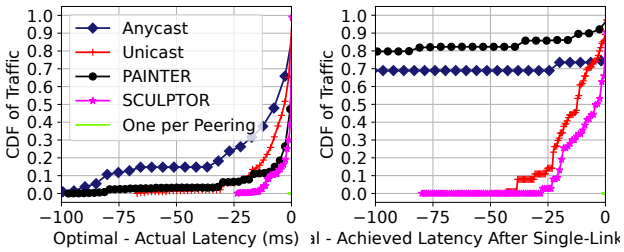
Latency + Maximum Link Utilization. Here we minimize average steady-state latency plus a constant × max-link-utilization (MLU). We set that constant equal to 0.1. We also set hard constraints that link utilization (computed from the sum of all UG traffic arriving on each link) does not exceed 1.0.

Dynamic Traffic Failover. This objective considers optimizing Latency + Maximum Link Utilization but also during single-link and single-site failures. The objective is thus the sum of Latency + Maximum Link Utilization on both the full deployment and sub-deployments with individually removed sites and links. This objective assumes that the Service Provider can rebalance traffic quickly using a technology like MPQUIC, and so permits different traffic distributions for each failure.

Static Traffic Failover. This objective is the same as Dynamic Traffic Failover, but assumes that prefix assignments are fixed during each link and site failure. This objective hence assumes that the Service Provider uses a traffic rebalancing technology like DNS which is too slow to quickly react to failure.

Latency Sensitive Services. Some services have very tight latency requirements (*e.g.,* ≤ 10 ms) [3]. This objective aims to maximize the amount of traffic within 10ms of its One-per-Peering latency. (We choose the difference with One-per-Peering latency as opposed to absolute latency to isolate SCULPTOR's goal of minimizing routing inefficiency.)

Traffic Classes. Similar to private WAN traffic engineering [18, 17], we consider routing traffic with distinct performance requirements. We consider a scenario where one traffic class (high-priority) should be routed with low latency, and the other (low-priority) should not congest the high-priority traffic. An additional challenge in the interdomain setting is that we cannot use priority queueing to ensure that high-priority traffic is not congested since we do not control queue behavior on the path. The objective is a weighted combination of the average latency of high-priority traffic and the amount of high-priority traffic that is congested. We also set a limit on the maximum overload on any link so that not all low-priority traffic lands on one link, as this solution would lead to poor low-priority goodput.



(a) (b)

Figure 5: SCULPTOR outperforms other solutions on the Internet deployment, reducing steady-state latency (a) and congestion during link failures (b).

Traffic Cost Across Sites. We also consider the scenario where each site has a cost proportional to the traffic volume arriving at the site. The cost per unit can vary across sites. This objective is a simple model of scenarios where Service Providers want to minimize power/carbon/peering infrastructure cost (which can vary by region) or minimize the use of unreliable sites (*e.g.,* dirty power). We minimize the weighted sum of average latency and average site cost. Our goal is to demonstrate that our approach supports trading off across these types of goals, not to argue that our simple cost model is correct.

5.2 On-Internet Experiments

We evaluate (a) Latency + Maximum Link Utilization and (b) Dynamic Traffic Failover on our Internet deployment (**??**). For context, at the scale of Service Providers that serve trillions of requests per day, improving a few percent of traffic by tens of milliseconds represents a significant improvement. Service Providers recently emphasized that small percentage gains are important [99, 100].

5.2.1 Steady-State Latency and MLU

Figure 5a shows a CDF of the difference in latency between each solution and One-per-Peering for all UGs, weighted by traffic. SCULPTOR outperforms other solutions and is only 1.9 ms worse than (the unreasonably expensive) One-per-Peering solution on average, closing 65% of the gap left by the next best solution, PAINTER, which is 5.5 ms worse. SCULPTOR also serves 91.8% of traffic within 10 ms of the latency with which One-per-Peering serves it, whereas the same is true for only 88.0% of traffic for PAINTER.

5.2.2 Failure Robustness

Figure 5b demonstrates that SCULPTOR offers lower latency paths for more UGs during single link failure in realistic routing conditions. For the users whose ingress failed, SCULPTOR is 7.9 ms higher latency than One-per-Peering and keeps 64.5% of their traffic within 10 ms of it, compared to unicast, the only prior approach that does not overload traffic, at 14.2 ms and 38.2%. PAINTER and anycast struggle to find sufficient capacity, overloading 75.8% and 69.0% of affected traffic.

Site failures yield similar results (not shown): SCULPTOR is 13.1 ms higher latency than One-per-Peering with 69.0% of affected traffic within 10 ms, while unicast is 25.1 ms higher with 39.6%. PAINTER again performs poorly, overloading 95.0% of traffic.

5.3 Emulated Experiments

We extensively evaluate SCULPTOR against other solutions across all objectives in our emulated deployments. Table 1 demonstrates that SCULPTOR outperforms all other solutions on deployments. Each objective was trained and computed on 10 emulated deployments, each with a random set of users and emulated routing conditions (**??**).

5.3.1 Latency + Maximum Link Utilization

Average latency for SCULPTOR is on average 1.3 ms worse than One-per-Peering, while the next-best solution (PAINTER) is a further 3.3 ms worse than SCULPTOR and has 10% higher latency. Interestingly, unicast (5.1 ms worse than One-per-Peering) performs better than AnyOpt (17.1 ms worse than One-per-Peering), which may be because AnyOpt was designed to minimize latency without capacity constraints while unicast naturally provides many high-capacity paths to users. SCULPTOR obtains the same maximum link utilization as One-per-Peering (0.967 vs 0.966 of anycast's).

5.3.2 Dynamic Traffic Failover

We next assess SCULPTOR's ability to robustly handle failures where the Service Provider has a technology for rebalancing traffic across prefixes quickly after the failure (*e.g.,* using MPQUIC). Failures here could represent excessive DDoS traffic on the link/site (thus using the link/site as a sink for the attack), physical failure, resource draining, changes in latency on a path, congestion in the public Internet, and planned maintenance. Table 1 demonstrates that SCULPTOR shifts traffic without overloading alternate links/sites more effectively than other solutions, *without reactive BGP changes* which could cause further failure (§2.3).

In solving for traffic allocations during failure scenarios, links may be overloaded. We say all traffic arriving on a congested link is congested. We separately note the fraction of traffic that lands on congested links and do not include it in average latency computations, but say such traffic does *not* satisfy 10 ms, 50 ms, or 100 ms objectives.

Performance Under Ingress/Site Failure. We fail single ingress links and dynamically allocate traffic after the failures (*e.g.,* using a technology such as MPQUIC). We find that most solutions have zero congestion since, with 32 sites, there are many backup options that the Service Provider can redirect traffic to. However, oftentimes users will find this capacity only at a high-latency option: for the users whose ingress failed, SCULPTOR is 6.1 ms worse than One-per-Peering on average, while the next-best solution (PAINTER) is 16.7 ms worse and unicast is 21.0 ms worse.

Site failures yield more congestion across options since many backups are removed. SCULPTOR congests 5.14% of traffic on average, versus 7.31% for unicast and 11.79% for PAINTER, and its affected users are 2.8 ms worse than One-per-Peering versus 23.9 ms for PAINTER and 27.7 ms for unicast.

Flash Crowd and Diurnal Traffic Resilience

In training for failure robustness, we find that SCULPTOR enables distributing higher-than-normal load over existing infrastructure. We quantify this improved infrastructure utilization under two realistic traffic patterns — flash crowds and diurnal patterns *without explicitly optimizing for these traffic patterns*.

We identify each client with a single "region" corresponding to the site at which they have the lowest possible latency ingress link. We scale UG traffic volumes until some link overloads in two different ways: (a) in one region at a time (a flash crowd) and (b) on a diurnal peak/trough cycle sampled from a Service Provider's published traffic [20] (shown in Appendix C). In **??** we report the critical scale at which overloading occurs relative to anycast's. SCULPTOR absorbs a flash crowd 26% larger than anycast and a diurnal swing 32% larger, against 1.22× and 1.26× anycast's for unicast, the next-best solution.

5.3.3 Static Traffic Failover

Under these evaluations we also fail links and sites but *do not* allow dynamic reallocation—BGP recomputes routes and users route to their next-best preferred option. Hence, SCULPTOR benefits from, but does not require, dynamic traffic rebalancing to retain its benefits.

Under ingress failure SCULPTOR's latency stays within 4.8 ms of One-per-Peering with 0.04% of traffic congested, whereas PAINTER, the next-best solution, is 11.0 ms worse than One-per-Peering with 0.19% congested. Under site failure SCULPTOR is 4.1 ms worse than One-per-Peering and congests 3.94% of traffic, 40% less than the best prior approach (AnyOpt, 6.55%) and 46% less than PAINTER (7.32%), which is 8.5 ms worse than One-per-Peering.

Objectives	Latency + Maximum Link Utilization (§§ 5.2.1 and 5.3.1)		Dynamic Traffic Failover (§§ 5.2.2 and 5.3.2)			Static Traffic Failover (§5.3.3)		Latency Sensitive Services (§5.3.4)	Traffic Classes (§5.3.5)		Traffic Cost Across Sites (§5.3.6)
Method	Latency (ms)	Maximum link utilization (vs anycast)	Ingress fail: latency (ms) / % cong	Site fail: latency (ms) / % cong	Flash Crowd/Diurnal Intensity (vs anycast)	Ingress fail: latency (ms) / % cong	Site fail: latency (ms) / % cong	% beyond 10ms of optimal	HPrio latency (ms)	Crit bulk ratio	Wgt avg site cost (vs anycast)
One-per-Peering [1]	29.1	0.966	29.3 / 0.00	30.8 / 5.55	1.37 / 1.44	29.2 / 0.00	32.1 / 0.50	12.9	29.0	5.62	0.918
SCULPTOR	**30.4**	**0.967**	**30.4 / 0.00**	**32.2 / 5.14**	**1.26 / 1.32**	**34.0 / 0.04**	**36.2 / 3.94**	**16.3**	**31.3**	**4.35**	**0.941**
PAINTER	33.7	0.995	34.1 / 0.00	36.5 / 11.79	1.04 / 1.08	40.3 / 0.19	40.6 / 7.32	21.0	33.7	1.29	0.980
unicast	34.2	0.978	34.9 / 0.00	36.7 / 7.31	1.22 / 1.26	51.9 / 0.38	49.7 / 6.58	24.0	34.2	2.74	0.953
AnyOpt	46.2	0.999	46.2 / 0.26	35.0 / 22.17	1.00 / 1.04	48.4 / 0.16	46.9 / 6.55	31.8	46.1	1.19	0.992
anycast	51.2	1.000	51.8 / 0.37	49.6 / 6.63	1.00 / 1.00	51.9 / 0.38	49.7 / 6.58	32.7	51.7	0.25	1.000

[1] An unrealistic optimal, included for comparison.

Table 1: Performance of all methods across objectives. SCULPTOR outperforms all other methods on every metric.

5.3.4 Latency Sensitive Services

We also evaluate SCULPTOR's ability to enable latency-sensitive services that have hard latency requirements such as VR/AR with a requirement of 10 ms [3]. Specifically, we try to maximize the amount of traffic to within 10 ms of its lowest latency option. This objective is highly non-linear (a step function) and so is challenging to optimize.

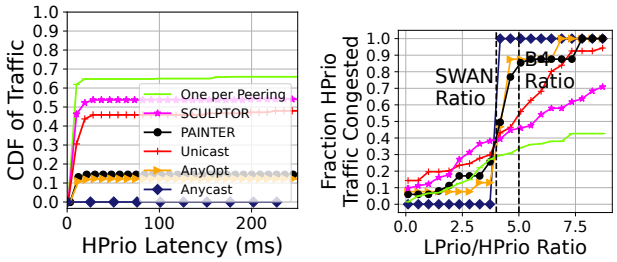
Table 1 shows that SCULPTOR brings 83.7% of traffic within 10 ms of its One-per-Peering latency, 4.7 points more than PAINTER (79.0%), leaving 22% less traffic beyond the threshold, and, compared to anycast, pushes us 83% of the way to the One-per-Peering solution (87.1%).

5.3.5 Traffic Classes

We also evaluate SCULPTOR's ability to satisfy multiple traffic classes. We split traffic into high and low-priority. The objective is to route high-priority traffic to low-latency routes while limiting congestion on those routes from low-priority traffic. We do not penalize cases where low-priority is congested, but do limit the maximum amount of any traffic on a link to 10× the capacity of the link to avoid solutions where all low-priority is placed on one link, as this solution would lead to low goodput for low-priority traffic (in practice, UGs would lower sending rates in response to congestion). We solve SCULPTOR on a single emulated 32 site deployment.

In Figure 6b we vary the amount of low-priority traffic as a multiple of high-priority traffic volume and compute the fraction of high-priority traffic congested. Links are provisioned to 5× the high-priority traffic volume (different from Section 5.3.2), as that is roughly the LPrio/HPrio ratio reported in prior work [18, 17]. There is insufficient global capacity to route all traffic for all LPrio/HPrio over 4.0, thus the jump in anycast congestion in Figure 6b. SCULPTOR finds strategies that allow us to route more low-priority traffic with less congestion than all other approaches.

Across the deployments in Table 1, SCULPTOR routes 4.35× the high-priority volume in bulk traffic before high-priority traffic congests, 1.6× the next-best solution (unicast, 2.74×) and well above PAINTER (1.29×), against 5.62× for One-per-Peering, while keeping high-priority latency within 2.3 ms of One-per-Peering (PAINTER: 4.7 ms).



(a) (b)

Figure 6: SCULPTOR routes high-priority traffic with low latency (6a) and minimal congestion from low-priority traffic (6b). We inferred LPrio/HPrio ratios for SWAN [18] and B4 [17] from those papers.

5.3.6 Traffic Cost Across Sites

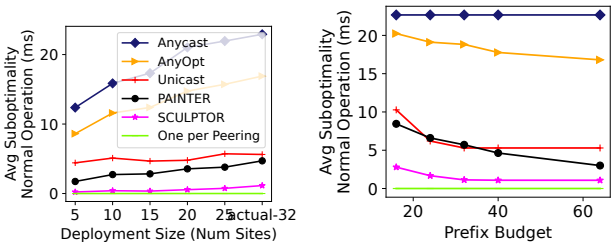
We also evaluate SCULPTOR's ability to jointly minimize both latency and per-site traffic cost, since different sites may cost more or less to serve traffic due to, for example, the availability of power. We solve SCULPTOR on three emulated 32 site deployments(**??**), using different per-site costs: (a) completely random per-site costs, (b) a model of expensive vs cheap sites where half the sites were 1.2× the cost of other sites, and (c) a model of per-site cost based on the grams of CO_2 emitted per kWh of electricity consumed at the site, similar to prior work [101, 102]. We do not claim to have realistic site cost values but use different site-cost choices to demonstrate that SCULPTOR's benefits are not specific to a configuration, and results were similar for each option so we report averages.

SCULPTOR finds advertisements that route traffic to lower-cost sites. On the deployments in Table 1, SCULPTOR's traffic-weighted site cost is 0.941× anycast's, 6% lower and 72% of the way to One-per-Peering (0.918×), versus 0.953× for unicast and 0.980× for PAINTER.

5.4 Micro-Evaluations

5.4.1 Ablation Study

SCULPTOR consists of several methodological changes over prior work (AnyOpt, PAINTER) that enable it to achieve its



(a) (b)

Figure 7: Micro evaluations.

goals. All of them aim to solve Equation (1), but model that objective function and search through the space in different ways. To break down how much each change matters, we conduct an ablation study where we randomly emulated 10 10-site deployments and optimized `Dynamic Traffic Failover` (§5.1.2) with different solvers.

Methodology. First, we evaluate `PAINTER` which uses a routing preference model and estimates objectives as the average objective value over ingresses with greedy search. Then we evaluate "Monte Carlo" which replaces objective estimation with our methodology in Section 3.4.1. Then we evaluate with "Monte Carlo + descent" which is the descent outlined in Section 3.5 without the gradient/memory. Then we evaluate "Monte Carlo + gradient descent" and, finally, we add in smart measurements which is the final component of `SCULPTOR` (§3.4.2).

Results. Normalizing `PAINTER` to 0% and `One-per-Peering` to 100%, we find that (on average), Monte Carlo gives 40% of the benefit, Monte Carlo + descent gives 78% of the benefit, and both Monte Carlo + gradient descent and full `SCULPTOR` (with smart measurements) give 91% of the benefit. Smart measurements, however, allow us to achieve the same benefit with 10% fewer measurements (*i.e.,* converging quicker), on average.

5.4.2 Effect of Deployment Size & Prefix Budget

We vary deployment size and prefix budget separately for at least 10 deployments at each budget/size while optimizing `Dynamic Traffic Failover`. We are primarily interested if the difference between `SCULPTOR` and the baselines grows, shrinks, or holds as these parameters vary. Figure 7 shows how steady-state latency varies—results are similar for all other metrics.

Deployment Size For each size, we select a random subset of all sites and emulate a deployment of that size using all ingresses at that site. We find that all metrics generally *hold*, and grow significantly compared to `PAINTER`. For example, at 32 sites `SCULPTOR`'s steady-state latency is 1.1 ms above `One-per-Peering` against 4.7 ms for `PAINTER`, nearing `unicast` performance; `SCULPTOR`'s lead grows 58% from the smallest to the largest deployment and it leads at every size (6/6).

Prefix Budget For each prefix budget, `SCULPTOR` leads every baseline, and its advantage is the largest when prefix budget is small: its steady-state lead over `PAINTER` shrinks from 5.7 ms at the smallest budget to 1.9 ms at the largest, its ingress-failure lead from 27.7 to 7.7 ms, and its site-failure congestion lead from 34.3 to 11.1 points.

6 Related Work

Egress Traffic Engineering. Prior work noted that traffic from Service Providers to users sometimes experienced suboptimal performance due to BGP's limitations [83, 35, 85]. `SCULPTOR` works in tandem with these systems, similarly working with BGP, but to engineer ingress traffic. Other work also shifts traffic to other links/sites to lower cost [84, 59, 20]. `SCULPTOR` adapts this idea in a new way but instead uses the public Internet to carry traffic to lower costs.

Ingress Traffic Engineering. We evaluate (§5) and discuss (§2.3) differences to other ingress traffic engineering [46, 82, 16, 48, 49, 6, 1, 103]. `FastRoute` coarsely balances users across anycast rings to respond to changing conditions [5], giving Service Providers much less control over the specific path users take. PECAN [104] and Tango [105] exhaustively expose paths between endpoints and so may provide resilience to changing conditions if the right paths were exposed. However, exposing all the paths does not scale to our setting since there are too many paths to measure. Recent work balances ingress traffic across sites during failure [87], or optimizes latency by predicting traffic how traffic routes [106, 103], but it is unclear how to extend these methodologies to other objectives.

Other prior work built a BGP playbook to mitigate DDoS attacks [86], but it is unclear if those strategies scale to larger Service Providers (they tested on a few sites/providers). `SCULPTOR` works for Service Providers with thousands of peering links. Other work/companies create overlay networks and balance load through paths in these overlay networks to satisfy latency requirements [107, 108, 109, 110, 38]. `SCULPTOR` works alongside these overlay networks by advertising the external reachability of these nodes to create better paths through the overlay structure.

Intradomain Failure Planning. Service Providers have shown interest in reducing the frequency/impact of failures in their global networks [111, 32, 33, 15]. `SCULPTOR` works alongside these systems by, for example, enabling Service Providers to shift traffic away from failed components/regions while still retaining good performance. Prior work also shifted traffic during peak times to limit cost/congestion [20], but did so using a private backbone. `SCULPTOR`'s benefits are orthogonal to this prior work and useful for Service Providers without private backbones, as they use the public Internet to realize the same result. Other prior work tried to plan intradomain routes to minimize the impact of k-component failures [97, 98, 112, 113, 20]. `SCULPTOR` solves different challenges that arise due to the lack of visibility/control into potential

paths and their capacities in the interdomain setting.

7 Discussion & Conclusions

Unilateral control over intradomain traffic has enabled technology that improves performance, reliability, and flexibility, including fast reroute for quick recovery from failure [114], virtual routing and forwarding for flexible traffic engineering [115], and virtual output queueing for differentiated service [23]. Conversely, interdomain proposals for differentiated services (*e.g.,* intserv/diffserv [116], L4S [117]) and multipath routing (MIRO [118]) have seen little deployment.

SCULPTOR requires no Internet-wide changes. Instead, it leverages black-box modeling and BGP flexibility to unlock a provider's unused global capacity. SCULPTOR is a step towards providing Service Providers with programmable interdomain networking primitives enabling the performance that modern services demand.

References

[1] Thomas Koch, Shuyue Yu, Ethan Katz-Bassett, Ryan Beckett, and Sharad Agarwal. [n.d.]. PAINTER: Ingress Traffic Engineering and Routing for Enterprise Cloud Networks. In *SIGCOMM 2023*.

[2] Bhaskar Kataria, Palak LNU, Rahul Bothra, Rohan Gandhi, Debopam Bhattacherjee, Venkata N Padmanabhan, Irena Atov, Sriraam Ramakrishnan, Somesh Chaturmohta, Chakri Kotipalli, et al. 2024. Saving Private WAN: Using Internet Paths to Offload WAN Traffic in Conferencing Services. *arXiv preprint arXiv:2407.02037* (2024).

[3] Nitinder Mohan, Lorenzo Corneo, Aleksandr Zavodovski, Suzan Bayhan, Walter Wong, and Jussi Kangasharju. [n.d.]. Pruning Edge Research With Latency Shears. In *HotNets 2020*.

[4] Jan-Philipp Stauffert, Florian Niebling, and Marc Erich Latoschik. 2018. Effects of latency jitter on simulator sickness in a search task. In *2018 IEEE conference on virtual reality and 3D user interfaces (VR)*.

[5] Ashley Flavel, Pradeepkumar Mani, David Maltz, Nick Holt, Jie Liu, Yingying Chen, and Oleg Surmachev. [n.d.]. FastRoute: A Scalable Load-Aware Anycast Routing Architecture for Modern CDNs. In *NSDI 2015*.

[6] Michael Markovitch, Sharad Agarwal, Rodrigo Fonseca, Ryan Beckett, Chuanji Zhang, Irena Atov, and Somesh Chaturmohta. [n.d.]. TIPSY: Predicting Where Traffic Will Ingress a WAN. In *SIGCOMM 2022*.

[7] Giovane C. M. Moura, Ricardo de Oliveira Schmidt, John Heidemann, Wouter B. de Vries, Moritz Muller, Lan Wei, and Cristian Hesselman. [n.d.]. Anycast vs. DDoS: Evaluating the November 2015 Root DNS Event. In *IMC 2016*.

[8] Matthias Wichtlhuber, Eric Strehle, Daniel Kopp, Lars Prepens, Stefan Stegmueller, Alina Rubina, Christoph Dietzel, and Oliver Hohlfeld. 2022. IXP scrubber: learning from blackholing traffic for ML-driven DDoS detection at scale. In *Proceedings of the ACM SIGCOMM 2022 Conference*. 707–722.

[9] Duncan Riley. 2023. Internet's rapid growth faces challenges amid rising denial-of-service attacks. https://siliconangle.com/2023/09/26/internets-rapid-growth-faces-challenges-amid-rising-ddos-attacks

[10] Giovane CM Moura, John Heidemann, Moritz Müller, Ricardo de O. Schmidt, and Marco Davids. [n.d.]. When the dike breaks: Dissecting DNS defenses during DDoS. In *IMC 2018*.

[11] Matthew Prince. 2013. The DDoS That Knocked Spamhaus Offline (And How We Mitigated It). https://blog.cloudflare.com/the-ddos-that-knocked-spamhaus-offline-and-ho

[12] Sam Kottler. 2018. February 28th DDoS Incident Report. https://github.blog/2018-03-01-ddos-incident-report/

[13] Chris Stokel-Walker. 2021. Case study: How Akamai weathered a surge in capacity growth. https://increment.com/reliability/akamai-capacity-growth-surge/

[14] Satyajeet Singh Ahuja, Varun Gupta, Vinayak Dangui, Soshant Bali, Abishek Gopalan, Hao Zhong, Petr Lapukhov, Yiting Xia, and Ying Zhang. [n.d.]. Capacity-efficient and uncertainty-resilient backbone network planning with hose. In *SIGCOMM 2021*.

[15] Bingzhe Liu, Colin Scott, Mukarram Tariq, Andrew Ferguson, Phillipa Gill, Richard Alimi, Omid Alipourfard, Deepak Arulkannan, Virginia Jean Beauregard, Patrick Conner, et al. [n.d.]. CAPA: An Architecture For Operating Cluster Networks With High Availability. In *NSDI 2024*.

[16] Xiao Zhang, Tanmoy Sen, Zheyuan Zhang, Tim April, Balakrishnan Chandrasekaran, David Choffnes, Bruce M. Maggs, Haiying Shen, Ramesh K. Sitaraman, and Xiaowei Yang. [n.d.]. AnyOpt: Predicting and Optimizing IP Anycast Performance. In *SIGCOMM 2021*.

[17] Sushant Jain, Alok Kumar, Subhasree Mandal, Joon Ong, Leon Poutievski, Arjun Singh, Subbaiah Venkata, Jim Wanderer, Junlan Zhou, Min Zhu, et al. [n.d.]. B4: Experience with a Globally-Deployed Software Defined WAN. In *SIGCOMM 2013*.

[18] Chi-Yao Hong, Srikanth Kandula, Ratul Mahajan, Ming Zhang, Vijay Gill, Mohan Nanduri, and Roger Wattenhofer. [n.d.]. Achieving High Utilization with Software-Driven WAN. In *SIGCOMM 2013*.

[19] Congcong Miao, Zhizhen Zhong, Yunming Xiao, Feng Yang, Senkuo Zhang, Yinan Jiang, Zizhuo Bai, Chaodong Lu, Jingyi Geng, Zekun He, et al. 2024. MegaTE: Extending WAN Traffic Engineering to Millions of Endpoints in Virtualized Cloud. (2024).

[20] David Chou, Tianyin Xu, Kaushik Veeraraghavan, Andrew Newell, Sonia Margulis, Lin Xiao, Pol Mauri Ruiz, Justin Meza, Kiryong Ha, Shruti Padmanabha, et al. [n.d.]. Taiji: Managing Global User Traffic for Large-Scale Internet Services at the Edge. In *SOSP 2019*.

[21] Brandon Schlinker, Todd Arnold, Italo Cunha, and Ethan Katz-Bassett. [n.d.]. PEERING: Virtualizing BGP at the Edge for Research. In *CoNEXT 2019*.

[22] VULTR. 2023. VULTR Cloud. https://vultr.com/

[23] Juniper Networks. 2023. Understanding CoS Virtual Output Queues (VOQs). https://juniper.net/documentation/us/en/software/junos/traffic-mgmt-qfx/topics/concept/cos-qfx-series-voq-understanding.html

[24] Mordor Intelligence. 2022. Network as a Service Market - Growth, Trends, COVID-19 Impact, and Forecasts. https://mordorintelligence.com/industry-reports/network-as-a-service-market-growth-trends-and-forecasts

[25] Krishan Arora. 2023. The Gaming Industry: A Behemoth With Unprecedented Global Reach. https://forbes.com/sites/forbesagencycouncil/2023/11/17/the-gaming-industry-a-behemoth-with-unprecedented-global-reach

[26] Cloudflare. 2024. Cloudflare Workers. https://workers.cloudflare.com/

[27] Fastly. 2024. Fastly Compute. https://fastly.com/products/edge-compute

[28] Edgecast. 2020. The CDN Edge brings Compute closer to where it is needed most. https://edgecast.medium.com/the-cdn-edge-brings-compute-closer-to-where-it-is-needed-most-d4a3f4107b11

[29] Google Cloud. 2024. Cloud Video Intelligence API. https://cloud.google.com/video-intelligence

[30] Microsoft. 2024. Emirates Global Aluminiium cuts cost of manufacturing AI by 86 percent with the introduction of Azure Stack HCL. https://customers.microsoft.com/en-us/story/1777264680029793974-ega-azure-arc-discrete-manufacturing-en-united-arab-emirates

[31] Yuchen Jin, Sundararajan Renganathan, Ganesh Ananthanarayanan, Junchen Jiang, Venkata N Padmanabhan, Manuel Schroder, Matt Calder, and Arvind Krishnamurthy. [n.d.]. Zooming in on wide-area latencies to a global cloud provider. In *SIGCOMM 2019*.

[32] Ramesh Govindan, Ina Minei, Mahesh Kallahalla, Bikash Koley, and Amin Vahdat. [n.d.]. Evolve or die: High-availability design principles drawn from googles network infrastructure. In *SIGCOMM 2016*.

[33] Umesh Krishnaswamy, Rachee Singh, Nikolaj Bjørner, and Himanshu Raj. [n.d.]. Decentralized cloud wide-area network traffic engineering with BLASTSHIELD. In *NSDI 2022*.

[34] Jeffrey C Mogul, Rebecca Isaacs, and Brent Welch. [n.d.]. Thinking about availability in large service infrastructures. In *HotOS 2017*.

[35] Brandon Schlinker, Hyojeong Kim, Timothy Cui, Ethan Katz-Bassett, Harsha V. Madhyastha, Italo Cunha, James Quinn, Saif Hasan, Petr Lapukhov, and Hongyi Zeng. [n.d.]. Engineering Egress with Edge Fabric: Steering Oceans of Content to the World. In *SIGCOMM 2017*.

[36] Wes Davis. 2023. Netflix ends a three-year legal dispute over Squid Game traffic. https://theverge.com/2023/9/18/23879475/netflix-squid-game-sk-broadband-partnership

[37] Amogh Dhamdhere, David D Clark, Alexander Gamero-Garrido, Matthew Luckie, Ricky KP Mok, Gautam Akiwate, Kabir Gogia, Vaibhav Bajpai, Alex C Snoeren, and Kc Claffy. 2018. Inferring persistent interdomain congestion. In *Proceedings of the 2018 Conference of the ACM Special Interest Group on Data Communication*. 1–15.

[38] Jinyang Li, Zhenyu Li, Ri Lu, Kai Xiao, Songlin Li, Jufeng Chen, Jingyu Yang, Chunli Zong, Aiyun Chen, Qinghua Wu, et al. [n.d.]. Livenet: a low-latency video transport network for large-scale live streaming. In *SIGCOMM 2022*.

[39] Yarin Perry, Felipe Vieira Frujeri, Chaim Hoch, Srikanth Kandula, Ishai Menache, Michael Schapira, and Aviv Tamar. [n.d.]. DOTE: Rethinking (Predictive) WAN Traffic Engineering. In *NSDI 2023*.

[40] Mark Jackson. 2020. Record Internet Traffic Surge Seen by UK ISPs on Tuesday. https://ispreview.co.uk/index.php/2020/11/record-internet-traffic-surge-seen-by-some-uk-isps-on-tuesday.html

[41] Martyn Landi. 2023. Return of original Fortnite map causes record traffic on Virgin Media O2 network. https://independent.co.uk/tech/fortnite-twitter-b2442476.html

[42] Scott Moritz. 2021. Internet Traffic Surge Triggers Massive Outage on East Coast. https://bloomberg.com/news/articles/2021-01-26/internet-outage-hits-broad-swath-of-eastern-u-s-customers

[43] Doug Madory. 2024. GP Leak Leads to Spike of Misdirected Traffic. https://kentik.com/analysis/BGP-Routing-Leak-Leads-to-Spike-of-Misdirected-Traffic

[44] Andrew Griffin. 2024. Facebook, Instagram, Messenger down: Meta platforms suddenly stop working in huge outage. https://independent.co.uk/tech/facebook-instagram-messenger-down-not-working-latest-b2507376.html

[45] Doug Madory. 2024. Outage Notice From Microsoft. https://twitter.com/DougMadory/status/1768286812894605517

[46] Matt Calder, Ashley Flavel, Ethan Katz-Bassett, Ratul Mahajan, and Jitendra Padhye. [n.d.]. Analyzing the Performance of an Anycast CDN. In *IMC 2015*.

[47] Thomas Koch, Ethan Katz-Bassett, John Heidemann, Matt Calder, Calvin Ardi, and Ke Li. [n.d.]. Anycast in Context: A Tale of Two Systems. In *SIGCOMM 2021*.

[48] Jiangchen Zhu, Kevin Vermeulen, Italo Cunha, Ethan Katz-Bassett, and Matt Calder. [n.d.]. The Best of Both Worlds: High Availability CDN Routing Without Compromising Control. In *IMC 2022*.

[49] Minyuan Zhou, Xiao Zhang, Shuai Hao, Xiaowei Yang, Jiaqi Zheng, Guihai Chen, and Wanchun Dou. [n.d.]. Regional IP Anycast: Deployments, Performance, and Potentials. In *SIGCOMM 2023*.

[50] Patrick Shuff. [n.d.]. Building a Billion User Load Balancer. https://usenix.org/conference/lisa16/conference-program/presentation/shuff

[51] Rupa Krishnan, Harsha V. Madhyastha, Sridhar Srinivasan, Sushant Jain, Arvind Krishnamurthy, Thomas Anderson, and Jie Gao. [n.d.]. Moving Beyond End-to-End Path Information to Optimize CDN Performance. In *IMC 2009*.

[52] Fangfei Chen, Ramesh K. Sitaraman, and Marcelo Torres. [n.d.]. End-User Mapping: Next Generation Request Routing for Content Delivery. In *SIGCOMM 2015*.

[53] Zhihao Li, Dave Levin, Neil Spring, and Bobby Bhattacharjee. [n.d.]. Internet Anycast: Performance, Problems, & Potential. In *SIGCOMM 2018*.

[54] Ann Bednarz. 2023. Global Microsoft cloud-service outage traced to rapid BGP router updates. https://networkworld.com/article/971873/global-microsoft-cloud-service-outage-traced-to-rapid-bgp-router-updates.html

[55] Santosh Janardhan. 2021. More details about the October 4 outage. https://engineering.fb.com/2021/10/05/networking-traffic/outage-details/

[56] Ben Cartwright-Cox. 2023. Grave flaws in BGP Error handling. https://edgecast.medium.com/the-cdn-edge-brings-compute-closer-to-where-it-is-needed-most-d4a3f4107b11

[57] Maxime Piraux, Louis Navarre, Nicolas Rybowski, Olivier Bonaventure, and Benoit Donnet. [n.d.]. Revealing the evolution of a cloud provider through its network weather map. In *IMC 2022*.

[58] Leon Poutievski, Omid Mashayekhi, Joon Ong, Arjun Singh, Mukarram Tariq, Rui Wang, Jianan Zhang, Virginia Beauregard, Patrick Conner, Steve Gribble, et al. [n.d.]. Jupiter evolving: transforming google's datacenter network via optical circuit switches and software-defined networking. In *SIGCOMM 2022*.

[59] Rachee Singh, Sharad Agarwal, Matt Calder, and Paramvir Bahl. [n.d.]. Cost-Effective Cloud Edge Traffic Engineering with Cascara. In *NSDI 2021*.

[60] NANOG. 2022. Panel: Buying and Selling IPv4 Addresses. https://youtube.com/watch?v=8FlTJEct9_s

[61] Geoff Huston. 2025. How we Measure: IPv6. https://www.potaroo.net/ispcol/2025-11/measure-ipv6.html

[62] Geoff Huston. 2025. IP Addresses through 2025. https://www.potaroo.net/ispcol/2026-01/addr2025.html

[63] 2026. Google IPv6 Statistics. https://www.google.com/intl/en/ipv6/statistics.html

[64] RIPE NCC Staff. 2023. RIS Live. (2023). https://ris-live.ripe.net

[65] Ticiane Takami. 2021. Project Myriagon: Cloudflare Passes 10,000 Connected Networks. https://blog.cloudflare.com/10000-networks-and-beyond/

[66] Pavlos Sermpezis and Vasileios Kotronis. 2019. Inferring catchment in internet routing. (2019).

[67] Microsoft. 2023. Microsoft Datacenters. https://datacenters.microsoft.com/

[68] Todd Arnold, Jia He, Weifan Jiang, Matt Calder, Italo Cunha, Vasileios Giotsas, and Ethan Katz-Bassett. [n.d.]. Cloud Provider Connectivity in the Flat Internet. In *IMC 2020*.

[69] Costin Raiciu, Christoph Paasch, Sebastien Barre, Alan Ford, Michio Honda, Fabien Duchene, Olivier Bonaventure, and Mark Handley. [n.d.]. How Hard Can It Be? Designing and Implementing a Deployable Multipath TCP. In *NSDI 2012*.

[70] Quentin De Coninck and Olivier Bonaventure. [n.d.]. Multipath QUIC: Design and Evaluation. In *CoNEXT 2017*.

[71] Quentin De Coninck and Olivier Bonaventure. 2021. Multiflow QUIC: A generic multipath transport protocol. *IEEE Communications Magazine* (2021).

[72] Patrick Sattler, Juliane Aulbach, Johannes Zirngibl, and Georg Carle. [n.d.]. Towards a Tectonic Traffic Shift? Investigating Apple's New Relay Network. In *IMC 2022*.

[73] Matt Calder, Ryan Gao, Manuel Schröder, Ryan Stewart, Jitendra Padhye, Ratul Mahajan, Ganesh Ananthanarayanan, and Ethan Katz-Bassett. [n.d.]. Odin: Microsoft's Scalable Fault-Tolerant CDN Measurement System. In *NSDI 2018*.

[74] Wouter B. De Vries, Ricardo de O. Schmidt, Wes Hardaker, John Heidemann, Pieter-Tjerk de Boer, and Aiko Pras. [n.d.]. Broad and Load-Aware Anycast Mapping with Verfploeter. In *IMC 2017*.

[75] Palak Goenka, Kyriakos Zarifis, Arpit Gupta, and Matt Calder. 2022. Towards client-side active measurements without application control. *SIGCOMM CCR 2022* (2022).

[76] [n.d.]. Shannon Entropy. https://en.wikipedia.org/wiki/Entropy_(information_theory)

[77] Sophie Langer. 2021. Approximating smooth functions by deep neural networks with sigmoid activation function. *Journal of Multivariate Analysis 2021* (2021).

[78] Yoav Shechtman, Amir Beck, and Yonina C Eldar. 2014. GESPAR: Efficient phase retrieval of sparse signals. *IEEE transactions on signal processing* (2014).

[79] Yurii Nesterov et al. 2018. *Lectures on convex optimization*. Vol. 137. Springer.

[80] Haochuan Li, Jian Qian, Yi Tian, Alexander Rakhlin, and Ali Jadbabaie. 2024. Convex and non-convex optimization under generalized smoothness. *Advances in Neural Information Processing Systems* 36 (2024).

[81] SciPy developers. 2024. SciPy linprog (HiGHS). https://docs.scipy.org/doc/scipy/reference/optimize.linprog-highs.html

[82] Hongqiang Harry Liu, Raajay Viswanathan, Matt Calder, Aditya Akella, Ratul Mahajan, Jitendra Padhye, and Ming Zhang. [n.d.]. Efficiently Delivering Online Services over Integrated Infrastructure. In *NSDI 2016*.

[83] Kok-Kiong Yap, Murtaza Motiwala, Jeremy Rahe, Steve Padgett, Matthew Holliman, Gary Baldus, Marcus Hines, Taeeun Kim, Ashok Narayanan, Ankur Jain, et al. [n.d.]. Taking the Edge off with Espresso: Scale, Reliability and Programmability for Global Internet Peering. In *SIGCOMM 2017*.

[84] Zheng Zhang, Ming Zhang, Albert G. Greenberg, Y. Charlie Hu, Ratul Mahajan, and Blaine Christian. [n.d.]. Optimizing Cost and Performance in Online Service Provider Networks. In *NSDI 2010*.

[85] Raul Landa, Lorenzo Saino, Lennert Buytenhek, and João Taveira Araújo. [n.d.]. Staying Alive: Connection Path Reselection at the Edge. In *NSDI 2021*.

[86] ASM Rizvi, Leandro Bertholdo, João Ceron, and John Heidemann. [n.d.]. Anycast Agility: Network Playbooks to Fight DDoS. In *USENIX Security Symposium 2022*.

[87] Jiangchen Zhu, Tom Koch, Ilgar Mammadov, Shuyue Yu, Kevin Vermeulen, Matt Calder, Italo Cunha, and Ethan Katz-Bassett. [n.d.]. The New (Pareto) Frontier of Cloud Routing: High Availability, Precise Control, or Configuration Stability-Choose Two. *CoNEXT 2025*.

[88] RIPE NCC Staff. 2015. RIPE Atlas: A Global Internet Measurement Network. *Internet Protocol Journal* (2015).

[89] Nikolaos Chatzis, Georgios Smaragdakis, Anja Feldmann, and Walter Willinger. 2013. There is more to IXPs than meets the eye. *SIGCOMM Computer Communication Review* (2013).

[90] Geoff Huston. 2014. How Big is that Network? labs.apnic.net/?p=526

[91] Loqman Salamatian, Calvin Ardi, Vasileios Giotsas, Matt Calder, Ethan Katz-Bassett, and Todd Arnold. [n.d.]. What's in the Dataset? Unboxing the APNIC per AS User Population Dataset. In *IMC 2024*.

[92] Danilo Giordano, Danilo Cicalese, Alessandro Finamore, Marco Mellia, Maurizio Munafò, Diana Zeaiter Joumblatt, and Dario Rossi. [n.d.]. A First Characterization of Anycast Traffic from Passive Traces. In *TMA 2016*.

[93] Danilo Cicalese, Jordan Augé, Diana Joumblatt, Timur Friedman, and Dario Rossi. [n.d.]. Characterizing IPv4 Anycast Adoption and Deployment. In *CoNEXT 2015*.

[94] Verizon. 2020. Edgecast. https://verizondigitalmedia.com/media-platform/delivery/network/

[95] Matthew Prince. 2013. Load Balancing without Load Balancers. https://blog.cloudflare.com/cloudflares-architecture-eliminating-single-p/

[96] Jing'an Xue, Weizhen Dang, Haibo Wang, Jilong Wang, and Hui Wang. [n.d.]. Evaluating Performance and Inefficient Routing of an Anycast CDN. In *International Symposium on Quality of Service 2019*.

[97] Ye Wang, Hao Wang, Ajay Mahimkar, Richard Alimi, Yin Zhang, Lili Qiu, and Yang Richard Yang. [n.d.]. R3: resilient routing reconfiguration. In *SIGCOMM 2010*.

[98] Hongqiang Harry Liu, Srikanth Kandula, Ratul Mahajan, Ming Zhang, and David Gelernter. [n.d.]. Traffic engineering with forward fault correction. In *SIGCOMM 2014*.

[99] Tobias Flach, Nandita Dukkipati, Andreas Terzis, Barath Raghavan, Neal Cardwell, Yuchung Cheng, Ankur Jain, Shuai Hao, Ethan Katz-Bassett, and Ramesh Govindan. [n.d.]. Reducing web latency: the virtue of gentle aggression. In *SIGCOMM 2013*.

[100] David Wetherall, Abdul Kabbani, Van Jacobson, Jim Winget, Yuchung Cheng, Charles B Morrey III, Uma Moravapalle, Phillipa Gill, Steven Knight, and Amin Vahdat. [n.d.]. Improving Network Availability with Protective ReRoute. In *SIGCOMM 2023*.

[101] Electricity Maps. 2026. Electricity Maps. https://www.electricitymaps.com/data

[102] Sawsan El-Zahr, William Nathan, and Noa Zilberman. [n.d.]. Carbon-Intelligent Content Scheduling in CDNs. In *ANRW 2025*.

[103] Minyuan Zhou, Yuning Chen, Jiaqi Zheng, Yifei Xu, Pan Hu, Yongping Tang, Wendong Yin, Jie Lin, Qingyan Yu, Yuanchao Su, Guihai Chen, Wanchun Dou, Songwu Lu, and Wan Du. [n.d.]. AnyPro: Preference-Preserving Anycast Optimization based on Strategic AS-Path Prepending. In *NSDI 2026*.

[104] Vytautas Valancius, Bharath Ravi, Nick Feamster, and Alex C Snoeren. [n.d.]. Quantifying the Benefits of Joint Content and Network Routing. In *SIGMETRICS 2013*.

[105] Henry Birge-Lee, Sophia Yoo, Benjamin Herber, Jennifer Rexford, and Maria Apostolaki. [n.d.]. {TANGO}: Secure Collaborative Route Control across the Public Internet. In *NSDI 2024*.

[106] Minyuan Zhou, Jiaqi Zheng, Congying Wang, Guihai Chen, and Wanchun Dou. [n.d.]. AnyTuner: Optimizing IP Anycast Performance via Strategic BGP Routing Policy. In *ICNP 2025*.

[107] Subspace. 2022. Optimize Your Network on Subspace. https://subspace.com/solutions/reduce-internet-latency

[108] INAP. 2022. INAP Network Connectivity. https://inap.com/network/

[109] Akamai. 2022. Akamai Secure Access Service Edge. https://akamai.com/resources/akamai-secure-access-service-edge-sase

[110] Cloudflare. 2022. Argo Smart Routing. https://cloudflare.com/products/argo-smart-routing/

[111] John P Eason, Xueqi He, Richard Cziva, Max Noormohammadpour, Srivatsan Balasubramanian, Satyajeet Singh Ahuja, and Biao Lu. [n.d.]. Hose-based cross-layer backbone network design with Benders decomposition. In *SIGCOMM 2023*.

[112] Yiyang Chang, Chuan Jiang, Ashish Chandra, Sanjay Rao, and Mohit Tawarmalani. 2019. Lancet: Better network resilience by designing for pruned failure sets. (2019).

[113] Chuan Jiang, Sanjay Rao, and Mohit Tawarmalani. [n.d.]. PCF: provably resilient flexible routing. In *SIGCOMM 2020*.

[114] IETF. 2005. Fast Reroute Extensions to RSVP-TE for LSP Tunnels. https://datatracker.ietf.org/doc/html/rfc4090

[115] IETF. 2006. BGP/MPLS IP Virtual Private Networks (VPNs). https://datatracker.ietf.org/doc/html/rfc4364

[116] IETF. 2000. A Framework for Integrated Services Operation over Diffserv Networks. https://datatracker.ietf.org/doc/html/rfc2998

[117] IETF. 2023. Low Latency, Low Loss, and Scalable Throughput (L4S) Internet Service: Architecture. https://datatracker.ietf.org/doc/rfc9330/

[118] Wen Xu and Jennifer Rexford. [n.d.]. MIRO: Multi-Path Interdomain Routing. In *SIGCOMM 2006*.

[119] Sharad Agarwal, Chen-Nee Chuah, Supratik Bhattacharyya, and Christophe Diot. [n.d.]. Impact of BGP Dynamics on Router CPU Utilization. In *PAM 2004*.

A Methodology Clarifications

A.1 Definition of Terms

Table 2 includes a table of common terms and expressions used in Section 3, for ease of reference.

A.2 Objective Functions

Every objective we evaluate is an instance of Equation (2) and differs only in G. Throughout, $V = \sum_{\texttt{UG}} v(\texttt{UG})$ is total demand, the load on link l is $f_l(R,w) = \sum_{R(A)\,(p,\texttt{UG})=l} w(p,\texttt{UG})$, its utilization is $f_l/c(l)$, and traffic-weighted mean latency is $\Lambda(R,w) = \frac{1}{V}\sum_{p,\texttt{UG}} \mathcal{L}(\texttt{UG}, R(A)(p,\texttt{UG}))\, w(p,\texttt{UG})$. All are subject to the constraints of Equation (1).

Latency and MLU.

$$G = \Lambda(R,w) + \beta M \quad \text{s.t.} \quad \frac{f_l(R,w)}{c(l)} \le M \;\; \forall l \tag{3}$$

Minimizing forces M to the maximum link utilization; β sets the operator's tradeoff between propagation delay and headroom.

Dynamic traffic failover. Let $A_l = A * F_l$ denote the configuration with link or site l withdrawn, and let G_0 be Equation (3). Each failure scenario gets its *own* allocation $w^{(l)}$, modelling an operator who can rebalance traffic quickly (*e.g.,* over MPQUIC):

$$\min_{w} G_0(R(A),w) \;+\; \sum_{l} \alpha_l \min_{w^{(l)}} G_0(R(A_l), w^{(l)}) \tag{4}$$

Static traffic failover. The same term, with a *single* allocation shared across the nominal case and every failure, modelling an operator restricted to slow rebalancing (*e.g.,* DNS):

$$\min_{w} \Big[\, G_0(R(A),w) \;+\; \sum_{l} \alpha_l\, G_0(R(A_l),w) \,\Big] \tag{5}$$

Equation (5)'s feasible set is a restriction of Equation (4)'s, so for any A the static objective is never smaller than the dynamic one—an invariant we assert when generating results. We use $\alpha_l = 0.1$.

Latency-sensitive services. Let $\mathcal{L}^*(\texttt{UG}) = \min_l \mathcal{L}(\texttt{UG}, l)$ over ingresses reachable by UG, and $\tau = 10$ ms. We maximize traffic served within τ of that best option:

$$G = -\frac{1}{V}\sum_{p,\texttt{UG}} w(p,\texttt{UG}) \cdot \mathbf{1}[\,\mathcal{L}(\texttt{UG}, R(A)(p,\texttt{UG})) - \mathcal{L}^*(\texttt{UG}) \le \tau\,] \tag{6}$$

The indicator makes this a step function, and hence a harder search than the others.

Multiple traffic classes. Split demand into high-priority and bulk, with allocations w_H, w_B and loads f_l^H, f_l. With γ weighting congestion against latency and κ capping per-link overload:

$$G = \Lambda_H(R,w_H) + \gamma \frac{1}{V_H} \sum_{l:f_l > c(l)} f_l^H \quad \text{s.t.} \quad \frac{f_l}{c(l)} \le \kappa \;\; \forall l \tag{7}$$

The cap prevents degenerate solutions that pile all bulk traffic onto one link.

Per-site traffic cost. Site s serves traffic at unit cost κ_s, and $site(l)$ is the site terminating link l:

$$G = \Lambda(R,w) + \delta \frac{1}{V} \sum_{l} \kappa_{site(l)} f_l(R,w) \tag{8}$$

A.3 One Iteration

Putting the pieces together, one iteration of SCULPTOR proceeds as follows (Figure 3).

1. Threshold the relaxed configuration $\tilde{A}$ at 0.5 to obtain the current advertisement $mathit{A}$ (§3.5.2).
2. If the model's entropy is high, or enough iterations have passed since the last measurement, advertise A via BGP and observe the ingress each UG uses for each prefix compared to what was available to update the routing model (§3.4.2).
3. Draw K routing functions R from the current model, one ingress per UG and prefix (§3.4.1).
4. For a random subset of entries (p,l) of $\tilde{A}$, solve the traffic allocation with $A(p,l)$ set to 1 and to 0 under each sampled R (§3.6); the averaged differences, passed through a sigmoid, are the gradient entries.
5. Take a Nesterov step on $\tilde{A}$.
6. Repeat until the measurement budget is exhausted or the objective stops improving (§3.5.3).

B Considerations for Real-World Deployments

In Section 4 we discuss how we prototype SCULPTOR. Here we give a high-level overview of how a real cloud can practically, safely implement SCULPTOR. The measurement, advertisement, and learning process described in Section 3 is meant to be carried out before any production traffic is shifted to the prefixes SCULPTOR is configuring, as in prior work [46, 16, 1].

SCULPTOR requires latency measurements to each ingress. To obtain these, the Service Provider should direct clients to measure latency towards a test prefix using a measurement system similar to those found in the literature [73, 52, 74, 75]. Note that this process *does not involve production traffic*. The Service Provider should then advertise the test prefix to one ingress at a time and record client latencies to the ingress.

With these initial measurements, Service Providers can initialize the learning process described in Section 3.4.2 using

Variable	
A	Advertisement configuration over peers and prefixes
w	Traffic allocation to routes
R	Routing function, which maps advertisements to routes. Unknown at first but updated according to measurements
p	Prefix, one of many in a prefix budget (limited due to expenses)
l	Ingress link, Service Providers often have thousands
$c(l)$	Link l capacity
UG	User group, users that route similarly. *e.g.,*/24's or metros
$w(p,\texttt{UG})$	Amount of traffic from users in UG we direct towards prefix p
G	Objective function, *e.g.,* average latency
$v(\texttt{UG})$	Traffic volume for users UG
F_l	Failure on link l
β	Tradeoff parameter between minimizing latency and maximum link utilization
M	Maximum link utilization
α_l	Importance parameter for failure on link l, *e.g.,* if one failure is more likely than the other
Expression	
$l = R(A)(p,\texttt{UG})$	Route (ingress link l) resulting from A, for UG towards p
$\mathcal{L}(\texttt{UG},l)$	Latency for UG across l
$G(R(A),w)$	Objective function value under A and w

Table 2: Table of common terms and expressions.

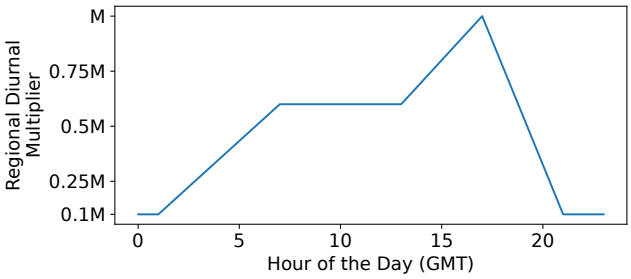


Figure 8: The diurnal traffic shape we apply in Section 5.3.2, taken from prior work [20].

test prefixes. Prior work also shows that BGP routing changes minimally disrupt routers [119], so SCULPTOR's learning process using test prefixes does not affect production traffic.

After convergence, SCULPTOR can slowly shift production traffic over to the optimal learned configuration. Note that, since most traffic is already routed along a good path [51, 46, 16, 1], SCULPTOR may only need to shift a small fraction of traffic or do so infrequently [6], mitigating the blast radius of configuration changes. The learning process is very infrequent, as prior work found that advertisements do not need to be updated even after a month [1].

C Diurnal Traffic Shape

In Figure 8, we show the shape of our diurnal curve taken from observed diurnal traffic patterns at a Service Provider [20]. We use this diurnal traffic pattern to evaluate how well balances traffic across sites (§5.3.2). Specifically, we linearly interpolate 6 points on the purple curve (Edge Node 1) in Figure 1 of that paper, capturing the essential peaks and troughs.